

Lab 1 – Data Link Protocol First and Second Classes Guide

2026/2027

First Lab Class

Objective

Exchange arrays of bytes between two computers using a serial cable and implement the establishment phase of the data link protocol.

Steps

1. Form groups of 2 elements per workbench

In NetLab (I322) there are 12 workbenches, numbered from 1-12. Please consider groups of 2 elements per workbench. The group members and the workbench should remain the same across the semester.

2. Select the correct computers

Each workbench has a rack of four computers (tux1 – tux4), and a separate small computer over the desk, closer to the centre of the room. In this work, we will use the serial port of this last computer and the one from tux2. To select tux2, you should press Scroll Lock + Scroll Lock + 2 + Enter on the keyboard (workbench 1-6) or press the number 2 on the KVM switch. The username and password are written on the workbench desk.

3. Ensure the serial port connectivity between the computers

To check the connectivity, open GTKTerm on both computers of the workbench and write some characters. The text should appear on the other computer. If this test fails, please check if the selected serial port is the correct one and that the baudrate is the same on both computers. Please report if the problem persists.

There are complete guides in Moodle explaining how the serial port works and how to use GTKTerm.

Note: the GTKTerm is only used to check the connectivity between the PCs. It must be closed while the data link protocol is running. When you are testing your programs make sure you do not have any GTKTerm instance running.

4. Download and Set up the Project's Base Code

In Moodle, there is a code project (code.zip), which should be used to develop the entire Lab 1 project. Download the base code ZIP file and extract it to a directory of your choice. Analyse the structure of the code and read the documentation provided.

The code project contains a **main.c** file, which serves as the program's entry point. Analyse this file, but do not modify it. Each layer of the protocol is implemented in a dedicated file. Therefore, the project is structured as follows:

- **application_layer.h** and **application_layer.c**: Implementation of the application layer protocol. The application_layer.h file must not be modified.
- **link_layer.h** and **link_layer.c**: Implementation of the link layer protocol. The link_layer.h file must not be modified.
- **serial_port.h** and **serial_port.c**: Implementation of the serial port library. This layer is already implemented and should not be modified.

The same project code is used for both the transmitter and the receiver. To run the program in the terminal, we need to pass multiple parameters as command-line arguments. One of the parameters is the role of the program, which is either "tx" if the program runs as the transmitter, or "rx" if the program runs as the receiver. These parameters are processed in the main.c file and used in the appropriate internal modules.

Each of the two PCs of the workbench acts as either the transmitter or the receiver, and they communicate with each other through the serial port. Therefore, one PC should run the program in transmitter mode ("tx") while the other PC should run the program in receiver mode.

In addition to the program's source files, the code project contains the **cable.c** program. This auxiliary program can be used to emulate a serial port in one PC, which can be used to work on the project outside of NetLab. There are tutorials about how to use this program in Moodle. Finally, the project contains a **Makefile**, which can be used to compile and run the program and cable.c.

Note: PCs in the NetLab are shared across all students. Therefore, remember to save your files to a private storage space before leaving each class (e.g., USB flash drive, cloud storage, GitHub / GitLab repository).

5. Send data through the serial port

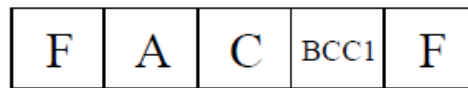
The code project comes with example code showing how to open a serial port in non-canonical mode and write / read data. The base code can be used as a starting point for this project. Read the link_layer.c file and analyse the example code to understand how the above operations are performed.

Compile the program in each PC (the PC acting as the transmitter and the PC acting as the receiver) using GCC or the Makefile. After compiling the program, run it in the terminal using the Makefile or writing the command directly in the terminal.

Note that one of the arguments of the program is the serial port name, which is `/dev/ttyS0` for the computer at the workbench and `/dev/ttyUSB0` for the computer at the desk. When running the example code, an array with the alphabet (abc...xyz) is sent through the serial port and printed on the receiver terminal.

6. Implement the logic connection establishment

Change the base code to send an array of bytes, defined in hexadecimal (declared as unsigned char), which performs the connection establishment procedure. This phase considers the exchange of supervision frames, which are composed of 5 bytes, and start with a flag, followed by an address field, a control field, a BCC and ending with another flag. The structure of the supervision frame is shown below.



Each field is defined as an unsigned char, which represents a byte in C language, and has the following values:

FLAG:

01111110 (0x7E)

A (Address):

00000011 (0x03) – frames sent by the Sender or answers from the Receiver

00000001 (0x01) – frames sent by the Receiver or answers from the Sender

C (Control):

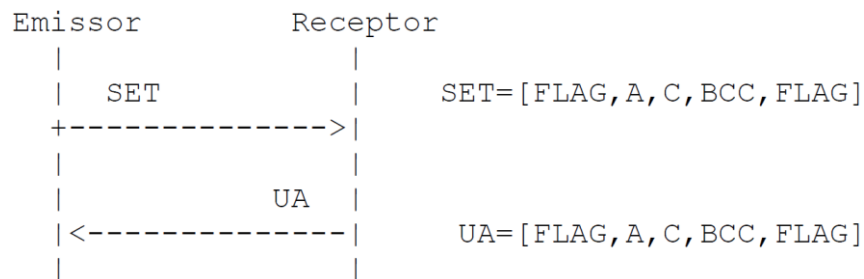
SET: 00000011 (0x03)

UA: 00000111 (0x07)

BCC1 (Block Check Character):

XOR of all characters of the header (in this case A and C)

The establishment phase is complete once the transmitter sends a SET frame, the receiver answers with an UA frame after checking that the received frame was correct, and the transmitter correctly detects the UA frame.



For debugging purposes, you can print the hexadecimal values using:

```
printf("var = 0x%02X\n", var);
```

If the above line does not work, due to older versions of the compiler, use instead:

```
printf("var = 0x%02X\n", (unsigned int)(var & 0xFF));
```

Please report to the Professor once this step is completed.